

Waple 업무일지 MCP 서버 — 프로젝트 상세 설명

8주 인턴 팀 프로젝트에서 담당한 MCP 서버 개발의 전체 과정을 정리한 문서입니다. 완성된 결과보다 어떤 판단을 했고 어디서 막혔는지에 무게를 두었습니다.

1. 프로젝트 개요

항목	내용
기간	2026.06.01 ~ 2026.07.26 (8주)
형태	기업 연계 인턴 팀 프로젝트 (4인 + 멘토 1인)
도메인	HR SaaS 플랫폼의 업무일지 자동화
본인 역할	팀장 겸 MCP 서버 설계·개발 전담
기술 스택	Python, MCP Python SDK(FastMCP), requests, pytest, nginx, systemd
산출물	MCP 서버 1식, 연동 매뉴얼, 배포 운영 가이드, 테스트 16종

팀은 지표 설계·AI 리포트 파트와 MCP 파트로 나뉘어 진행하였으며, 본 문서는 제가 담당한 MCP 서버 파트만을 다룹니다.

2. 문제 정의

2-1. 현장에서 관찰한 문제

업무일지는 인사 평가의 기초 자료이지만, 실제로는 하루가 끝날 무렵 "오늘 뭐 했더라"를 떠올리며 작성하는 경우가 많습니다. 그 결과 "개발 진행함", "회의 참석" 같은 내용 없는 기록이 쌓이고, 인사 데이터로서의 가치를 잃습니다.

2-2. 착안점

개발자의 하루는 이미 어딘가에 기록되어 있습니다.

- Git 커밋 메시지와 변경 파일
- 작업 중 수정한 파일의 변경 규모
- Claude Code 세션 로그

이미 존재하는 근거가 있는데도 다시 손으로 쓰고 있다는 점에 주목하였습니다. 따라서 기록을 모아 초안을 만들고, 사람은 확인만 하는 구조를 목표로 잡았습니다.

2-3. 경계 설정

"자동으로 써주는 도구"는 위험합니다. 업무일지는 인사 평가 자료이므로, 사실이 아닌 내용이 자동 등록되면 개인에게 직접적인 불이익이 됩니다.

그래서 프로젝트 초기에 다음 원칙을 세우고 끝까지 지켰습니다.

확인되지 않은 내용은 쓰지 않는다. 승인되지 않은 내용은 등록하지 않는다.

이 원칙이 이후 모든 설계 판단의 기준이 되었습니다.

3. 요구사항 확정 과정

초기 구상은 범위가 넓었습니다. 데이터베이스 직접 조회, 성과 평가 연계, 이직 위험도 예측 등이 후보로 올라와 있었습니다.

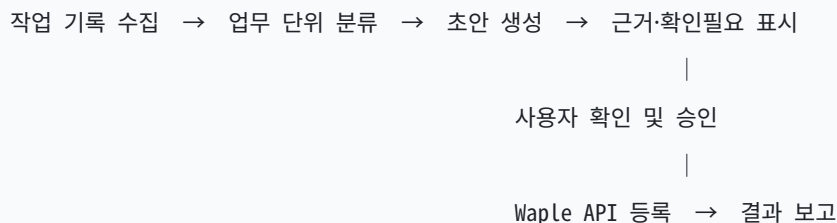
멘토 미팅을 거치며 다음과 같이 범위를 좁혔습니다.

구분	내용
포함	작업 기록 수집, 업무 단위 분류, 초안 생성, 승인 절차, API 등록, 결과 확인
제외	DB 직접 조회, 성과 평가, 이직 위험도 예측, 승인 없는 자동 등록

범위를 줄인 판단이 결과적으로 옳았습니다. 남은 8주 중 절반 이상이 "제대로 동작하는가"를 검증하는 데 들어갔기 때문입니다. 기능을 더 넣었다면 어느 것도 검증하지 못한 채 끝났을 것입니다.

4. 시스템 설계

4-1. 전체 흐름



승인 단계 이전에는 어떤 쓰기 요청(POST/PUT/DELETE)도 실행되지 않습니다.

4-2. 핵심 판단 ① — 초안 생성과 등록을 분리한다

프로젝트 지침에는 도구를 6개로 나누는 안이 제시되어 있었습니다. 실제로는 4개로 통합하였습니다.

지침안 (6개)	실제 구현 (4개)
<code>collect_work_activity</code>	<code>waple_login</code>
<code>summarize_daily_work</code>	<code>tasklog</code> (Git 기반 초안)
<code>create_worklog_draft</code>	<code>chat_tasklog</code> (대화 기반 초안)
<code>preview_worklog_request</code>	<code>submit_worklog</code> (등록)
<code>submit_worklog</code>	
<code>check_worklog_result</code>	

수집·요약·초안 생성·미리보기가 **한 번의 호출로 사용자에게 자연스럽게 노출되는** 구조라, 굳이 도구를 나눌 필요가 없었습니다. 지침 15번("필요한 기능만 최소한으로 구현")을 우선 적용한 판단입니다.

다만 **초안 생성과 등록만은 끝까지 분리**하였습니다. 하나로 합치면 "정리해줘"라는 말에도 API가 호출될 수 있기 때문입니다.

승인 표현도 명시적으로 구분하였습니다.

승인으로 본다	승인으로 보지 않는다
등록해 / 승인 / 이 내용으로 올려	정리해줘 / 초안 만들어줘 / 검토해줘 / 수정해줘

4-3. 핵심 판단 ② — 모든 문장에 근거를 붙인다

초안의 각 항목 끝에 **어디서 나온 내용인지**를 표시합니다.

- `test_connector.py` 의 `import` 문을 파일 상단으로 정리하였습니다. [커밋 97a822d9]
- 포트폴리오용 시연 자료를 촬영하였습니다. [사용자 메모]
- 오늘 사용한 토큰: 세션 로그 기준 집계 [Claude Code 세션 로그]

근거가 없는 내용은 본문에 넣지 않고 **"사용자 확인 필요"** 항목으로 분리합니다. 확인할 수 없는 작업을 사실처럼 쓰지 않기 위한 장치입니다.

토큰 집계가 불가능한 환경에서는 임의로 채우지 않고 **수집 불가 (Claude Code 세션 로그 미발견)** 이라고 그대로 표기합니다.

4-4. 핵심 판단 ③ — transport를 두 개 유지한다

방식	대상	인증
stdio	로컬 Claude Code	<code>.env</code> 파일의 키
Streamable HTTP	원격 접속 (nginx + HTTPS)	요청 헤더 <code>x-api-key</code>

원격 하나로 통일하려 했으나 불가능하였습니다. 원격 서버는 사용자 PC의 세션 로그(~/.claude/projects)와 Git 저장소에 접근할 수 없어 **근거 수집 자체가 되지 않기 때문**입니다.

기능을 포기하지 않으려면 두 경로를 모두 유지해야 했고, 이 제약은 지금도 구조적 한계로 남아 있습니다.

4-5. 핵심 판단 ④ — 원격에서는 사용자를 격리한다

원격 모드에서는 여러 사용자가 같은 서버 프로세스를 공유합니다. 초기 구현은 전역 변수를 쓰고 있어, 사용자 간 데이터가 섞일 수 있었습니다.

문제	조치
전역 변수에 키 저장	<code>ContextVar</code> 로 요청별 격리
초안 캐시가 전역 1개	키 해시(sha256 앞 16자리) 기준으로 사용자별 스코핑
원격 사용자 키가 서버 <code>.env</code> 에 기록됨	HTTP 모드에서는 <code>.env</code> 저장 자체를 차단

이 설계는 팀원의 코드 리뷰(PR #13)에서 지적을 받고 다시 만든 것입니다. 아래 6절에서 자세히 다룹니다.

5. 시행착오 — 코드 레벨

개발 중 확인하고 고친 문제들입니다. 대부분 코드 리뷰에서 발견되었습니다.

#	문제	원인	해결
1	등록 시 구분선과 '내일 계획'이 사라짐	승인 시점에 초안을 다시 만들면서 형식이 손실	전송용 원본을 캐시에 보관하고 등록 시 그대로 사용
2	한글 폴더명이 깨져서 표시	<code>git diff</code> 가 한글을 옥탈 이스케이프로 출력	<code>-c core.quotePath=false</code> 옵션 추가
3	<code>git add</code> 한 파일의 변경 규모가 누락	<code>git diff --numstat</code> 은 unstaged 만 집계	<code>git diff HEAD --numstat</code> 으로 변경
4	네트워크 오류 시 키 재검증이 조용히 넘어감	예외 처리에서 검증 블록을 건너뛸	재검증 실패 시 등록을 중단 하도록 변경
5	실패 후 재시도 시 초안이 어긋남	캐시 초기화 시점이 정해지지 않음	1회용 캐시로 재설계 — 성공 시에만 초기화, 실패 시 유지
6	토큰 집계가 누락되는 환경이 있음	로그 폴더명 대소문자를 구분해 매칭	소문자 비교 폴백 + 혼재 시 전체 합산

4번과 5번이 특히 기억에 남습니다.

4번은 "네트워크가 안 되면 검증을 건너뛰고 진행"이 편의처럼 보였지만, 실제로는 **검증 없이 등록되는 구멍**이었습니다. 안전한 쪽은 언제나 "확실하지 않으면 멈추는" 쪽입니다.

5번은 캐시를 지우면 재시도 시 원본이 사라지고, 안 지우면 낡은 내용이 등록되는 양쪽 위험이 있었습니다. "성공했을 때만 지운다"는 한 줄 규칙으로 두 문제가 동시에 해결되었습니다.

6. 시행착오 — 코드 리뷰

팀원이 리뷰어를 맡았고, PR 단위로 지적을 받아 수정하였습니다.

PR	지적 내용	조치
#6	재검증 fail-safe 누락, 캐시 설계 위험, staged diff 누락	3건 모두 수정 후 단위 테스트 추가
#7	로그인 실패 시 사용자가 다음에 뭘 해야 할지 모름	재시도 안내 문구 추가, 테스트 신규 작성
#12	로그 폴더 대소문자 처리 누락	폴백 로직 추가, 6케이스 테스트
#13	원격 모드 인증 설계 결함	구조 재설계 후 재리뷰 승인

PR #13 — 가장 크게 배운 지점

원격 접속 기능을 추가하면서, 저는 기존 stdio 구조를 거의 그대로 두고 transport만 바꾸었습니다. 동작은 하였습니다.

리뷰에서 지적받은 것은 **"동작하지만 안전하지 않다"** 는 점이었습니다. 여러 사용자가 같은 프로세스를 쓰는데 키와 초안이 전역 변수에 있었고, 원격 사용자의 키가 서버 파일에 기록될 수 있는 구조였습니다.

부분 수정으로 넘어갈 수도 있었지만, 요청별 격리 구조로 다시 만들었습니다. 결과적으로 코드가 늘었지만, **"동작한다"**와 **"맞게 만들었다"**가 다르다는 것을 분명하게 배운 경험이었습니다.

7. 시행착오 — 배포와 운영

코드가 완성된 뒤에 겪은 문제들입니다. 오히려 이쪽에서 시간을 더 많이 썼습니다.

7-1. 421 Misdirected Request

원격 배포 후 커넥터가 계속 421을 반환하였습니다. 서버는 정상 기동 중이고 포트도 열려 있었습니다.

원인은 FastMCP의 **DNS Rebinding 방지 기능**이었습니다. 허용 Host가 **127.0.0.1** 로 잡혀 있어, 외부 도메인으로 들어온 요청을 거부한 것입니다.

nginx에서 Host 헤더를 재작성해 우회하였습니다. 정석은 서버 코드에서 허용 Host를 명시하는 것이며, 이는 남은 과제로 두었습니다.

7-2. 인증서 체인 누락

브라우저에서는 접속이 잘 되는데, **원격 커넥터만 연결에 실패**하였습니다. Windows의 **curl** 도 성공했기 때문에 한동안 원인을 찾지 못했습니다.

`openssl s_client` 로 확인해 보니 인증서 체인에 **중간 인증서가 빠져** 있었습니다. 브라우저와 일부 클라이언트는 누락된 인증서를 자동으로 보완하지만, 리눅스 `curl` 과 서버 간 통신은 그렇지 않습니다.

"내 환경에서는 되는데"가 왜 위험한지 체감한 사례입니다. 서버 권한 밖의 문제라 담당자에게 상황을 정리해 전달하였고, fullchain 적용 후 해결되었습니다.

7-3. 프로세스 소멸

배포 초기에는 `nohup` 으로 백그라운드 실행하였습니다. 어느 날 서버가 502를 반환하기에 접속해 보니 프로세스가 사라져 있었습니다.

로그에 예외가 없었습니다. 즉 코드 오류가 아니라 **서버 재부팅으로 함께 종료**된 것입니다. `uptime` 이 18시간이었던 것으로 재부팅 시점도 확인하였습니다.

systemd 서비스로 전환하려 했으나, 계정의 `sudo` 권한이 nginx 관련 명령으로만 제한되어 있어 일반적인 시스템 레벨 등록이 불가능하였습니다.

루트 권한 없이 쓸 수 있는 **사용자 단위 systemd** 를 사용하고, `linger` 옵션을 켜서 로그인 여부와 무관하게 상시 동작하도록 하였습니다.

검증은 프로세스를 강제 종료(`kill -9`)한 뒤 PID가 바뀌는지로 확인하였습니다. 재부팅 검증은 하지 않았는데, 같은 서버에서 다른 팀의 API가 운영 중이라 임의 재부팅이 타 서비스 중단을 유발하기 때문입니다. 이 항목은 문서에도 "설정 확인, 재부팅 미실시"로 구분해 표기하였습니다.

7-4. 웹 커넥터는 끝내 불가능했다

claude.ai 웹에서 커스텀 커넥터로 등록은 되지만, 실제 도구 호출은 실패하였습니다.

서버 로그를 보니 도구 목록 조회(`ListToolsRequest`)까지는 정상이고, 실제 호출(`CallToolRequest`)이 아예 오지 않았습니다.

원인은 **제품 제약**이었습니다. 웹 UI 커넥터는 OAuth 방식만 지원하여 커스텀 헤더 입력란이 없고, 이 서버는 `x-api-key` 헤더로 인증합니다.

설정 파일을 직접 편집할 수 있는 Claude Code 와 데스크톱 앱에서는 정상 동작합니다. "안 되는 이유"를 로그로 특정할 수 있었던 점이 수확이었습니다.

8. 보안 사고와 대응

8-1. API 키가 터미널에 평문 출력됨

`.env` 에서 키를 찾아 로그인해 달라고 요청했더니, AI 도구가 `grep` 으로 **키 값 전체를 터미널에 출력**한 뒤 도구에 전달하였습니다.

키는 세 곳에 남았습니다.

- 터미널 스크롤백
- 세션 로그 파일
- 대화 기록

즉시 새 키를 발급하였는데, 여기서 한 번 더 배웠습니다. **Waple**은 다중 키 구조라 재발급만으로는 기존 키가 무효화되지 않았습니다. 반드시 기존 키를 삭제해야 했습니다.

이후 다음을 규칙으로 삼았습니다.

- 키가 필요한 작업은 AI에게 파일을 뒤지게 하지 않고 사용자가 직접 지정한다
- 부득이한 경우 "키 값은 화면에 출력하지 마"를 함께 지시한다
- 노출이 의심되면 재발급이 아니라 **삭제 후 재발급**한다

8-2. 외부 패키지가 헤더를 로그에 남김

데스크톱 원격 연동에 사용하는 `mcp-remote` 브리지는 전달받은 헤더를 표준 출력에 그대로 기록합니다. 우리 코드가 아니라 수정할 수 없습니다.

키를 인자에 직접 쓰지 않고 환경변수로 참조하도록 하여 프로세스 목록 노출은 막았지만, 로그 파일에는 여전히 남습니다. 이 사실을 매뉴얼에 명시하고, 화면 공유·시연 시 해당 로그를 캡처하지 않도록 안내하였습니다.

해결하지 못한 위험은 숨기지 말고 문서에 남긴다는 원칙을 적용한 사례입니다.

9. 테스트

9-1. 자동화 테스트

```
python -m pytest -q
# 16 passed
```

파일	검증 내용
<code>test_connector.py</code>	원격 커넥터, 사용자별 초안 스코핑
<code>test_cache.py</code>	초안 캐시 (성공 시 초기화 / 실패 시 유지)
<code>test_guideline17.py</code>	승인 절차, 필수값 누락, 중복 등록 방지
<code>test_login_retry.py</code>	인증 실패 시 재시도 안내
<code>test_token_usage.py</code>	세션 로그 기반 토큰 집계

9-2. 실환경 검증

자동화하기 어려운 항목은 실제 API를 호출해 확인하였습니다.

케이스	결과
유효한 키로 로그인	성공, <code>.env</code> 자동 생성, 키 마스킹 표시
잘못된 키(401)	실패 안내, 기존 키 유지
로그인 없이 등록 시도	"먼저 로그인하세요" 안내
삭제된 키로 재로그인	정상 거부
재검증 실패(네트워크 오류) 시 등록	등록 중단 메시지 출력
동일 날짜 재등록	덮어쓰기 안내 후 진행

일부 케이스는 다른 케이스와 동일한 코드 경로를 타므로, 실물 재현 대신 코드 리뷰로 대체 검증하였음을 문서에 명시하였습니다.

10. 검증 현황

사실과 추정을 구분하기 위해 항목마다 검증 수준을 표기하였습니다.

항목	결과	검증 수준
로컬(stdio) 전체 흐름	초안 생성 → 승인 → 등록 성공	실물 검증
원격(HTTP) 연동	호출 성공, 서버 로그에 기록 확인	실물 검증
데스크톱 앱 원격 연동	브리지 경유 연결 및 인증 성공	실물 검증
자동 재시작	강제 종료 후 PID 변경 확인	실물 검증
재부팅 후 자동 기동	설정값 확인 (<code>is-enabled</code> , <code>Linger=yes</code>)	설정 확인 (재부팅 미실시)
웹 커넥터	목록 조회까지만 가능, 실호출 불가	실물 검증

"동작할 것이다"와 "동작하는 것을 보았다"를 표에서 구분한 이유는, 프로젝트 후반에 이 구분이 없으면 스스로도 무엇을 확인했는지 알 수 없게 되기 때문입니다.

11. 배운 점

11-1. 정직함이 곧 기능이다

이 도구에서 가장 중요한 기능은 요약 성능이 아니라 **"모르는 것을 모른다고 말하는 것"**이었습니다.

커밋이 없는 날에는 초안이 비어 나옵니다. 처음에는 실패처럼 보였지만, 없는 일을 지어내지 않았다는 점에서 정상 동작이었습니다. 인사 자료를 다루는 도구라면 이쪽이 맞습니다.

11-2. 동작하는 것과 맞게 만든 것은 다르다

PR #13에서 배운 내용입니다. 원격 기능은 처음부터 "동작"은 했습니다. 문제는 여러 사용자가 쓸 때였습니다. 리뷰가 없었다면 그대로 넘어갔을 것입니다.

11-3. 내 환경에서 되는 것은 근거가 아니다

인증서 체인 문제에서 배웠습니다. 브라우저와 Windows `curl` 에서 성공했기 때문에 오히려 원인을 늦게 찾았습니다. 클라이언트마다 동작이 다르다는 전제를 갖고 봐야 했습니다.

11-4. AI 도구에 권한을 넘길 때는 범위를 정해줘야 한다

키 노출 사고는 도구가 잘못된 것이 아니라, 제가 범위를 정해주지 않은 결과였습니다. "알아서 해줘"는 편하지만, 무엇을 하면 안 되는지까지 알려주지 않으면 예상하지 못한 방식으로 처리됩니다.

11-5. 제약은 우회할 수 있다

배포 단계에서 `sudo` 권한이 없어 막혔을 때, "권한을 달라고 요청한다" 외에 다른 길이 없다고 생각했습니다. 사용자 단위 `systemd` 라는 방법을 찾은 뒤로는, 막히면 먼저 **그 환경에서 허용된 범위**를 확인하는 습관이 생겼습니다.

12. 한계와 남은 과제

정직하게 남깁니다.

항목	내용
구조적 한계	원격 모드에서는 Git·세션 로그 접근이 불가능해 근거 수집이 제한됨
미완	서버 코드에 허용 Host를 명시해 nginx 우회 없이 처리
미완	초안 캐시가 계속 쌓이는 문제 (등록 후 정리 필요)
미검증	재부팅 후 자동 기동 (설정만 확인)
외부 제약	웹 UI 커넥터는 인증 방식 차이로 사용 불가

프로젝트 종료 후에도 개선을 이어갈 계획입니다.

13. 저장소

- 코드 및 문서: github.com/psy0635-ctrl/waple-worklog-mcp-portfolio
- 시연 영상과 검증 캡처는 저장소의 `assets/` 폴더에 포함되어 있습니다.

회사 도메인·서버 주소·포트 등 인프라 정보는 플레이스홀더로 대체하였습니다.